scalert_event — full documentation

ქართული ვერსია · Setup guide (Georgian)

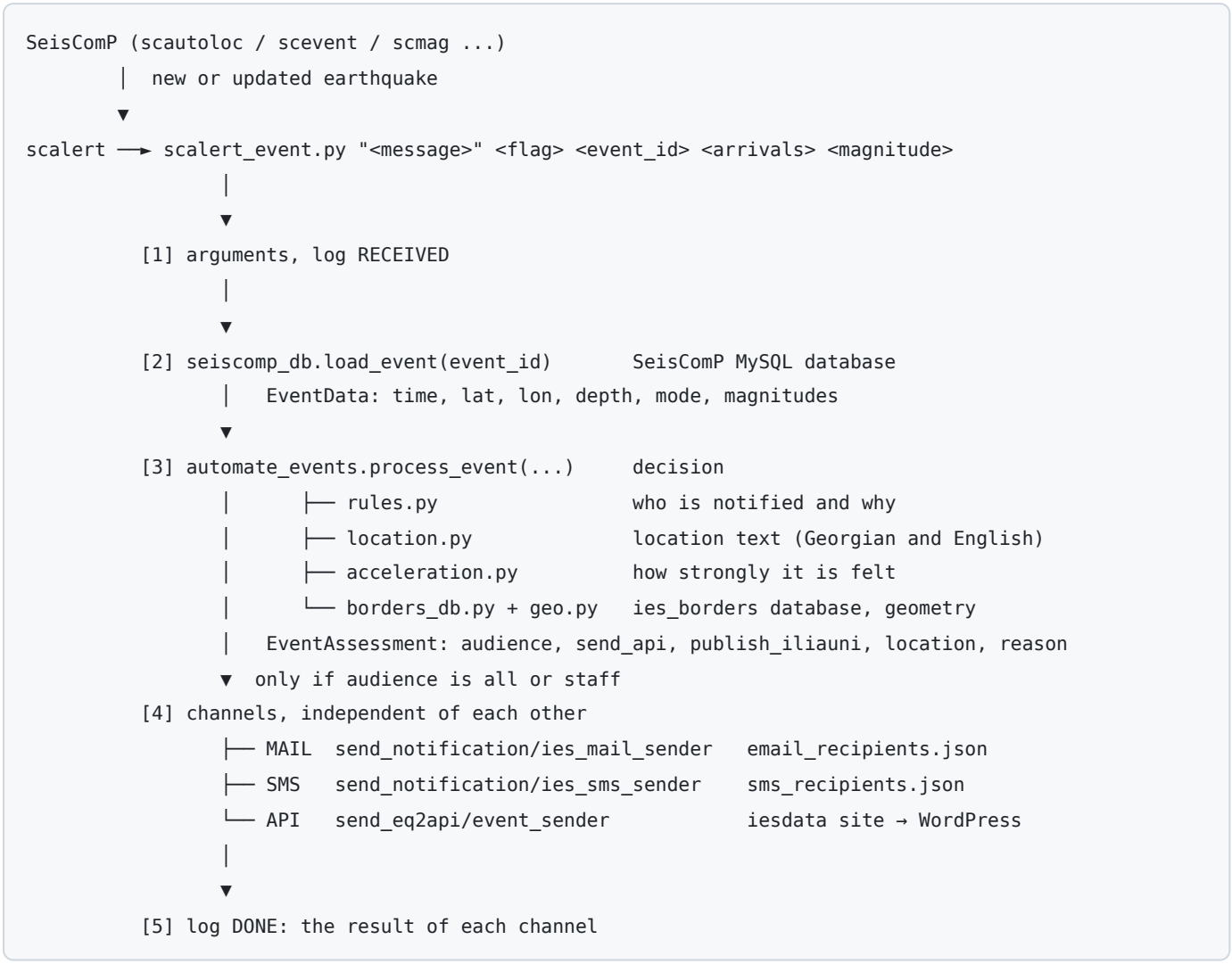
`scalert_event` sends automatic notifications about SeisComP earthquakes. `scalert` calls our script with an `event_id`. The script reads the earthquake parameters from the SeisComP database and decides, by rules, who gets notified. It then sends email and SMS, adds the earthquake to the iesdata site and, when required, publishes it on WordPress.

This document describes **how the system works**. Installation, filling in `.env` and troubleshooting are in the setup guide.

Contents

1. Overview
2. How scalert calls the script
3. Entry point: `scalert_event.py`
4. Data from the SeisComP database: `seiscomp_db.py`
5. Decision: `automate_events/`
6. Repeated recalculations of one earthquake
7. Email and SMS: `send_notification/`
8. iesdata site and WordPress: `send_eq2api/`
9. Log
10. Configuration: `.env`
11. Recipients
12. Tests
13. File layout

1. Overview



Principle: deciding and sending are separate. `automate_events` only decides and sends nothing. `send_notification` and `send_eq2api` only send and decide nothing. So a new rule goes only into `rules.py`, and a new channel (e.g. Telegram) becomes a separate module. Neither affects the other.

Channels do not stop each other. If one fails, the others are still sent, and the failure shows in the log and in the exit code.

2. How scalert calls the script

On every new earthquake and every update, `scalert` runs the script set in `scripts.event` with these arguments:

Argument	Meaning
`$1`	Text message (scalert format)

Argument	Meaning
`$2`	`1` — new earthquake, `0` — update
`$3`	Earthquake publicID, e.g. `ies2026rudf` — **this is the `event_id`**
`$4`	Number of arrivals (phases)
`$5`	Magnitude (may be missing)

The script relies only on `$3`. `$1`, `$2`, `$4` and `$5` are logged, but every parameter used for the decision is read from the database, so there is a single source of truth.

Running by hand (without scalert):

```
./scalert_event.py ies2026rudf
./scalert_event.py --dry-run ies2026rudf     # decides and logs, sends nothing
```

3. Entry point: `scalert_event.py`

It only connects the steps; there is no business logic here:

1. reads the arguments and logs `RECEIVED`;
2. `process()`: read from the database → decide → channels;
3. calls each channel separately; an error in one does not stop the others;
4. logs `DONE` and returns the exit code.

Code	Meaning
`0`	Processed: sent, or decided not to send
`1`	Error: the earthquake was not found in the database, or a channel failed with an error
`2`	Invalid arguments

A channel error is an API error (`EventApiError`), a recipients file error (`RecipientsError`) or any unexpected error. When SMTP or all SMS providers fail to send, this is logged (`MAIL FAILED`, `SMS FAILED`) but shows as `not_sent` in `DONE` and does not change the exit code.

4. Data from the SeisComP database: `seiscomp_db.py`

`load_event(event_id) -> EventData`

```
Event (publicID = event_id)
  ├── preferredOriginID     ──► Origin: time, latitude, longitude, depth, evaluationMode
  └── preferredMagnitudeID  ──► Magnitude (preferred)
Origin._oid ──► Magnitude._parent_oid: all magnitudes of this origin (ML, MLv, M, mb, ...)
```

- `Event.preferredOriginID` is used rather than the newest origin, because SeisComP itself considers the preferred origin the best one;
- SQL queries are parameterised (`cursor.execute(sql, (event_id,))`);
- **Race condition.** scalert gets the earthquake from messaging while `scdb` writes the database asynchronously, so the earthquake or its origin may not be in the database yet. `load_event` tries `SC_DB_RETRIES` times (default 5) with `SC_DB_RETRY_DELAY` seconds (3) between attempts and logs `WAITING`. If it is still missing, `EventNotReady` is raised and the log gets one line: `FAILED event_id=...: event ... not found`.

`EventData`:

Field	Type	Description
`event_id`	str	publicID
`origin_id`	str	publicID of the preferred origin
`origin_time`	datetime	UTC
`latitude`, `longitude`	float	degrees
`depth`	float	km; `0.0` if SeisComP has none
`evaluation_mode`	str	`automatic` / `manual`
`event_type`	str | None	`not existing`, `not locatable`, `outside of network interest` → `SKIPPED`
`magnitudes`	dict[str, float]	e.g. `{"ML": 3.4, "MLv": 3.3, "M": 3.4}`
`preferred_magnitude_type`	str | None	type of the preferred magnitude

5. Decision: `automate_events/`

`process_event(origin_time, lat, lon, depth, evaluation_mode, magnitudes) -> EventAssessment`

- checks that latitude, longitude and depth are numbers;
- time is UTC; +4 hours (Georgian time) is added for the message;
- magnitude choice: `ML` first, otherwise `M`, otherwise the first available; rounded to two decimals;

- opens a connection to the `ies_borders` database, calls `rules.decide`, closes it.

`EventAssessment`:

Field	Description
`audience`	`all`, `staff` or `None` (nobody)
`send`	`audience is not None` — email and SMS are sent
`send_api`	whether it is added to the iesdata site; `True` when any rule matched
`publish_iliauni`	whether it is published on WordPress; `True` only for `all`
`location_ge`, `location_en`	location text; only when something is sent
`ev_mode`, `magnitude_type`, `magnitude`, `local_time`	for the message text
`reason`	Georgian reason with numbers: why this decision was made

5.1 Rules: `rules.py`

Group	Who receives	`send_api`	`publish_iliauni`
`all`	staff and external recipients	True	True
`staff`	staff only; the text starts with `STAFF ONLY.`	True	False
nobody	—	False	False

Epicentre in Georgia:

Condition	Sent to
magnitude ≥ 3.0	all
in Tbilisi and 2.8 ≤ magnitude < 3.0	all
in Tbilisi and magnitude < 2.8	nobody
outside Tbilisi and 2.8 ≤ magnitude < 3.0	staff only
outside Tbilisi, magnitude < 2.8, felt in Tbilisi like M2.8 or more	all
outside Tbilisi, magnitude < 2.8, felt weakly in Tbilisi	nobody

Epicentre abroad:

Condition	Sent to
felt at the border like an M3.0 or more in Georgia	all
otherwise, but ≤ 100 km from the border and magnitude ≥ 3.0	staff only
anything else	nobody

The thresholds (2.8, 3.0, 100 km) are constants at the top of `rules.py`. `send_api` and `publish_iliauni` are properties of `Decision`, so their rule changes in one place.

5.2 "Felt like M2.8 / M3.0": `acceleration.py`

It computes the acceleration the earthquake causes at a specific place (the Tbilisi border or the nearest point of the Georgian border), taking distance and depth into account. This is compared with a "reference": an earthquake of the threshold magnitude that happened exactly at that place. If the real earthquake shakes the place no less than the reference does, the threshold is reached.

```
log10(a) = -4.4983 + 1.3209·M − 0.0656·M² − 0.0041·R − 0.5229·log10(R²)
```

`R` is the hypocentral distance in km (epicentral distance, depth and a 3 km pseudo-depth).

5.3 Location text: `location.py`

It is built only when something is sent, to avoid extra database queries. Every fragment is created in both languages at once (`Text(ge, en)`).

Case	Georgian	English
in Georgia, ≥ 3.0	`საქართველო. ქალაქი ბორჯომი, აღმოსავლეთით:11კმ. სოფელი კიმოთესუბანი:4კმ.`	`Georgia. 11 km east of Borjomi (city). Nearest settlement: Kimotesubani (village), 4 km.`
in Tbilisi	`საქართველო.თბილისი, ვაკე-საბურთალოს რაიონი.`	`Georgia. Tbilisi, Vake-Saburtalo district.`
outside Tbilisi, < 3.0	`საქართველო. თბილისიდან 61კმ. ქალაქი დმანისი, დასავლეთით:21კმ. სოფელი მამიშლარი:5კმ.`	`Georgia. 61 km from Tbilisi. 21 km west of Dmanisi (city). Nearest settlement: mamishlari (village), 5 km.`
abroad	`ტაო-კლარჯეთი, საქართველოს საზღვრიდან 85კმ. დაბა შუახევი, სამხრეთით:104კმ. სოფელი წელათი:90კმ.`	`Tao-Klarjeti, 85 km from the Georgian border. 104 km south of Suaxevi (town). Nearest settlement: Tselati (village), 90 km.`

- The Georgian text repeats the old system's format exactly, including spaces and dots;
- distance to a city = distance to its centre − city radius − 1 km. If the epicentre is inside the city, only the city is written;
- 1 km is subtracted from the settlement distance (old format), so the Georgian text can say `0ვ0` or `-1ვ0`; in English a negative distance becomes `0 km`;
- the direction is from the city towards the epicentre;
- city and settlement names in both languages come from the `ies_borders` database (`name_ge`, `name_en`, `status_ge`, `status_en`). Names of countries, Russian regions and Tbilisi districts are in `location.py`. A wrong English name in the database (e.g. `mamishlari`, `Suaxevi`) is fixed in the database; no code change is needed.

6. Repeated recalculations of one earthquake

scalert calls the script on **every** update: a new origin, a new magnitude, a manual review. Each call goes through all the steps independently. Fresh data is read from the database, the rules are evaluated again, and if the result meets the criteria the notification is sent, regardless of whether the previous recalculation was sent.

Channel	On every recalculation
email, SMS	a new message with the data of that moment; the previous one is not "cancelled"
iesdata site	the record with the same `seiscomp_oid` is **updated**; no new one is created
WordPress	`publish_iliauni=True` → published or updated; `False` and published before (e.g. `all` → `staff`) → unpublished

If two calls arrive at the same time, both are processed separately.

7. Email and SMS: `send_notification/`

File	What it does
`message.py`	`build_subject`, `build_body`, `build_sms_text` — text only
`recipients.py`	`recipient_emails(audience)`, `recipient_numbers(audience)` — recipients (section 11)
`mailer.py`	SMTP sending with the `MAIL_*` settings
`ies_mail_sender.py`	`send_event_mail(...)` — text + recipients + SMTP, with logging

File	What it does
`ies_sms_sender.py`	`send_event_sms(...)` and `SmsSender` — SMS through a chain of providers, with logging

7.1 Email

One message is sent to everyone, but recipients cannot see each other's addresses.

Subject:

```
Earthquake - Automatic: 2026-10-07 05:15:03. mag=3.0M
```

Body (`STAFF ONLY.` only for staff-only events):

```
STAFF ONLY.
Automatic: 2026-10-07 05:15:03. mag=3.0M. ტაო-კლარჯეთი, საქართველოს საზღვრიდან 85კმ. ...
https://www.google.com/maps/place/40.68,42.27
Shakemap: https://shorturl.at/ibvtX

https://ies.iliauni.edu.ge/?page_id=121



იღიას სახელმწიფო უნივერსიტეტი
დედამიწის შემსწავლელ მეცნიერებათა ინსტიტუტი და სეისმური მონიტორინგის ეროვნული ცენტრი
```

Log	Meaning
`MAIL SENDING`	count, addresses, subject
`MAIL SENT`	handed over to SMTP successfully
`MAIL NOT SENT`	no active address in the group
`MAIL FAILED`	SMTP error (traceback in the same log)
`MAIL DRY RUN`	dry run: the text is in the log, nothing is sent

7.2 SMS

The text is the first lines of the email, as in the old system:

```
STAFF ONLY.
Automatic: 2026-09-16 22:20:17. mag=2.92M. საქართველო. თბილისიდან 61კმ. ქალაქი დმანისი,
დასავლეთით:21კმ. სოფელი მამიშღარი:5კმ.
```

Numbers are normalised to `995XXXXXXXXX` (`+995 555 12-34-56` and `555123456` are both accepted; an invalid number is logged and skipped).

Sending goes through a chain of providers (`SMS_PROVIDERS`, default `magti,smsoffice,gsm`). Each next provider gets **only the numbers the previous one failed on**:

Provider	How	Success
`magti`	Magti one-way gateway, per number	response `0000-<id>`, and after 5 s tracking status `1`, `4` or `8`
`smsoffice`	smsoffice.ge API, per number, up to 3 attempts	`{"Success": true}`
`gsm`	GSM modem (iesresource), one request for all remaining numbers	HTTP 200

- A provider whose password or key is empty in `.env` is skipped;
- an unconfirmed Magti SMS goes on to the next provider, so rarely one SMS may arrive twice. For an alert this is better than missing it;
- smsoffice accepts up to 160 characters; the length is in the log as `length=`;
- after sending, the smsoffice balance is checked; below `SMSOFFICE_BALANCE_WARNING` the log gets `SMS BALANCE LOW`.

Log	Meaning
`SMS SENDING`	numbers, text and its length
`SMS magti/smsoffice/gsm ...`	result per number per provider
`SMS via <provider>: sent=N failed=M`	provider total
`SMS SENT`	sent to every number
`SMS NOT SENT`	no active number in the group
`SMS FAILED`	no provider could send to some numbers (they are listed)
`SMS BALANCE LOW`	few SMS left on smsoffice
`SMS DRY RUN`	dry run: nothing is sent

8. iesdata site and WordPress: `send_eq2api/`

File	What it does
`event_sender.py`	`send_event_api(...)` — SeisComP earthquake in the site's format, publish / unpublish, with logging
`send_eq2api.py`	`EventApiClient` — validation, create / update, rollback on failure, publish / unpublish; also a read-only command

8.1 What happens

```
send_api=True ?  ──no──► nothing
     │ yes
     ▼
add_event: is the same seiscomp_oid already on the site?
     ├── no  → POST earthquake, POST each magnitude (on failure the earthquake is deleted)
     └── yes → PUT earthquake, PUT/POST magnitudes
     │
     ▼
publish_iliauni=True ?  ── yes ──► POST /api/publish_events/publish/{id}
     │ no
     ▼
published before (is_published)?  ── yes ──► POST /api/publish_events/unpublish/{id}
```

- **All or nothing.** If any magnitude of a new earthquake cannot be added, the created earthquake is deleted so that it does not stay on the site without a magnitude. If the delete also fails, the error says which id has to be deleted by hand;
- **On update** each magnitude is tried separately and the failed ones are reported together. Magnitudes on the site that are no longer in the new data are not deleted;
- **Two keys.** Adding earthquakes uses `Send_eq2api_API_KEY` (`can_event_edit`); publish/unpublish uses `SeisComp_Publisher_API_KEY` (`can_event_publish`);
- if publish or unpublish fails, the earthquake stays in its previous state on the site and the next recalculation tries again.

8.2 What is sent

Magnitudes in SeisComP `{"ML": 3.512, "MLv": 3.43, "mb": 4.64, "mB": 4.9, "M": 3.512}` are sent to the site as:

```
{
  "seiscomp_oid": "ies2026ufhk",
  "origin_time": "2026-10-12 09:35:14",
  "latitude": 41.84595,
  "longitude": 43.55186,
  "depth": 0.0,
  "magnitudes": {"ML": 3.512, "MLV": 3.43, "MB": 4.64, "M": 3.512},
  "location_ge": "საქართველო. ქაღაქი ბორჯომი, აღმოსავლეთით:11კმ. სოფელი კიმოთესუბანი:4კმ.",
  "location_en": "Georgia. 11 km east of Borjomi (city). Nearest settlement: Kimotesubani (village), 4
km.",
  "is_automatic": false
}
```


Field	Source
`seiscomp_oid`	`event_id`
`origin_time`	UTC, to the second (`YYYY-MM-DD HH:MM:SS`)
`latitude` , `longitude`	5 decimals
`depth`	km, 3 decimals
`magnitudes`	site codes, 3 decimals (see below)
`location_ge` , `location_en`	section 5.3
`is_automatic`	`evaluation_mode == "automatic"`

Magnitude codes. The site knows: `ML, MB, MS, MD, MW, K, MPV, MLH, MC, MLV, M` (`SITE_MAGNITUDE_CODES`). SeisComP types match them case-insensitively (`MLv` → `MLV` , `mb` → `MB` , `Mw` → `MW`). Exceptions:

- `Ms_20` → `MS` ;
- `mB` (broadband body-wave) is **not sent**, because it is a different magnitude from `mb` ;
- types unknown to the site (e.g. `Mwp` , `mb_Lg`) are not sent and are logged.

When a new magnitude is added on the site, it must also be added to `SITE_MAGNITUDE_CODES` in `send_eq2api.py` .

8.3 API

Request	Purpose	Key
`POST /api/seismic_events/filter`	find by `seiscomp_oid`	edit
`POST /api/seismic_events/`	create	edit
`PUT /api/seismic_events/{id}`	update	edit

Request	Purpose	Key
`DELETE /api/seismic_events/{id}`	delete a failed create	edit
`POST /api/seismic_events/{id}/magnitudes`	add a magnitude	edit
`PUT /api/seismic_events/magnitudes/{id}`	change a magnitude	edit
`POST /api/publish_events/publish/{id}`	publish or update on WordPress	publisher
`POST /api/publish_events/unpublish/{id}`	remove from WordPress	publisher

Every request has the `X-API-Key` header and a timeout (`Send_eq2api_API_TIMEOUT`, 20 s). Full API description: `https://iesdata.iliauni.edu.ge:3030/swagger.json`.

Checking by hand (read-only):

```
./send_eq2api/send_eq2api.py ies2026ufhk
```

Log	Meaning
`API MAGNITUDES SKIPPED`	magnitude types unknown to the site that were skipped
`API SENDING`	the data to send and `publish_iliauni`
`API CREATED` / `API UPDATED`	added / updated on the site, with `site_id`
`API PUBLISHED`	published on WordPress
`API UNPUBLISHED`	removed from WordPress
`API NOT SENT`	no magnitude known to the site
`API DRY RUN`	dry run: nothing is sent
`API FAILED`	API error, one line

9. Log

File: `logs/event_scalert.log`. Line format: `time LEVEL KEYWORD event_id=... ...`

Every line has `event_id=...`, so the whole history of one earthquake is:

```
grep "event_id=ies2026rudf" logs/event_scalert.log
grep " API " logs/event_scalert.log        # site only
grep " ERROR " logs/event_scalert.log      # problems only
```

Keyword	Meaning
`RECEIVED`	scalert called the script, with its arguments
`WAITING`	the earthquake is not fully in the database yet; retrying
`INPUT`	data read from the database
`DECISION`	`SEND` or `NOT SENT`, with the Georgian reason
`MAIL ...`, `SMS ...`, `API ...`	channel details (sections 7 and 8)
`SKIPPED`	not processed (not a real earthquake, or no magnitude yet)
`DONE`	finished: the result of each channel (`sent` / `not_sent` / `failed`) and the duration; `ERROR` if any channel failed
`FAILED`	could not be processed

Every run ends with exactly one of: `SKIPPED`, `DECISION ... NOT SENT`, `DONE` or `FAILED`.

A traceback is written only for unexpected errors. An expected error is one line: an API response, a recipients file, or an earthquake not found in the database.

Example, two recalculations:

```
INFO RECEIVED event_id=ies2026rudf flag=new arrivals=12 magnitude=4.2 dry_run=False message='...'
INFO INPUT event_id=ies2026rudf origin=Origin/... origin_time=2026-09-08 04:52:15 lat=41.607
lon=43.5676 depth=15.3 mode=automatic type=earthquake magnitudes={'ML': 4.23} preferred_magnitude=ML
INFO DECISION event_id=ies2026rudf SEND to=all publish_iliauni=True send_api=True magnitude=4.23ML
local_time=2026-09-08 08:52:15 location=საქართველო. ქაღაქი ახაღქაღაქი, ... reason: ეპიცენტრი
საქართველოშია და მაგნიტუდა 4.23 ≥ 3.0
INFO MAIL SENDING event_id=ies2026rudf audience=all count=2 recipients=... subject='Earthquake -
Automatic: 2026-09-08 08:52:15. mag=4.23ML'
INFO MAIL SENT event_id=ies2026rudf audience=all count=2
INFO SMS SENDING event_id=ies2026rudf audience=all count=3 numbers=... length=119 text='...'
INFO SMS via magti: sent=3 failed=0
INFO SMS SENT event_id=ies2026rudf audience=all count=3
INFO API SENDING event_id=ies2026rudf publish_iliauni=True payload={...}
INFO API CREATED event_id=ies2026rudf site_id=12
INFO API PUBLISHED event_id=ies2026rudf site_id=12 message='Publish request completed.' wp_response='1'
INFO DONE event_id=ies2026rudf mail=sent sms=sent api=sent dry_run=False duration=7.4s
INFO RECEIVED event_id=ies2026rudf flag=update arrivals=15 magnitude=4.3 ...
...
INFO API UPDATED event_id=ies2026rudf site_id=12
INFO API PUBLISHED event_id=ies2026rudf site_id=12 ...
INFO DONE event_id=ies2026rudf mail=sent sms=sent api=sent dry_run=False duration=6.9s
```

Failure example:

```
ERROR API FAILED event_id=ies2026test: POST /api/seismic_events/ failed with HTTP 403: სერვისს არ აქვს საჭირო უფლება — Missing required permission: can_event_edit
ERROR DONE event_id=ies2026test mail=sent sms=sent api=failed dry_run=False duration=0.4s
```

A second log, `logs/update_recipients.log`, belongs to the recipients update (section 11).

10. Configuration: `.env`

All passwords, keys and addresses live only in `.env` (not committed to git); nothing is in the code. `config.py` reads `.env` and builds `Settings`. A process environment variable wins over `.env`, so a value can be changed for one run: `DRY_RUN=1 ./scalert_event.py ...`.

Variable	Default	Purpose
`SC_DB_HOST`, `SC_DB_USER`, `SC_DB_PASSWORD`, `SC_DB_NAME`	`127.0.0.1`, `sysop`, —, `seiscomp`	SeisComP database
`SC_DB_RETRIES`, `SC_DB_RETRY_DELAY`	`5`, `3`	how many times and how often to look for a not-yet-written earthquake
`BORDERS_DB_HOST`, `BORDERS_DB_USER`, `BORDERS_DB_PASSWORD`, `BORDERS_DB_NAME`	`127.0.0.1`, `ies_borders`, —, `ies_borders`	borders, cities and settlements database
`MAIL_SERVER`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`	—, `587`	SMTP
`SEND_EMAIL`, `SEND_SMS`, `SEND_EQ2API`	`1`	`0` — channel disabled
`RECIPIENTS_FILE`, `SMS_RECIPIENTS_FILE`	`recipients/email_recipients.json`, `recipients/sms_recipients.json`	recipient files
`Recp_Exporter_API_URL`, `Recp_Exporter_API_KEY`	iesdata	recipients API
`Recp_Exporter_API_CA`, `Recp_Exporter_API_TIMEOUT`	system CAs, `20`	only for a self-signed certificate; timeout
`Send_eq2api_API_URL`, `Send_eq2api_API_KEY`	iesdata	adding earthquakes (`can_event_edit`)

Variable	Default	Purpose
`SeisComp_Publisher_API_KEY`	—	publishing on WordPress (`can_event_publish`)
`Send_eq2api_API_CA` , `Send_eq2api_API_TIMEOUT`	system CAs, `20`	only for a self-signed certificate; timeout
`SMS_PROVIDERS` , `SMS_TIMEOUT`	`magti,smsoffice,gsm` , `10`	SMS provider order; timeout
`MAGTI_USERNAME` , `MAGTI_PASSWORD` , `MAGTI_CLIENT_ID` , `MAGTI_SERVICE_ID`	—, —, `643` , `1`	Magti
`SMSOFFICE_KEY` , `SMSOFFICE_SENDER` , `SMSOFFICE_BALANCE_WARNING`	—, `IES` , `5000`	smsoffice.ge
`GSM_SMS_URL` , `GSM_SMS_CODE`	iesresource, —	GSM modem
`LOG_FILE` , `LOG_LEVEL`	`logs/event_scalert.log` , `INFO`	log; with `DEBUG` the rule inputs are logged too
`DRY_RUN`	`0`	`1` — nothing is sent, log only

Relative paths (e.g. `recipients/...`) are resolved from the project folder.

11. Recipients

Recipients are managed on the iesdata Notify page. `recipients/update_recipients.py` builds two local files from the API (neither is committed to git):

```
iesdata API  GET /api/recips/  (X-API-Key)
     │   {"items": [{username, is_staff, is_active,
     │               emails:  [{email, is_active}],
     │               numbers: [{phone_number, is_active}]}], "total": N}
     ▼
recipients/update_recipients.py      (by cron, every 10 minutes)
     │                     │
     ▼                     ▼
email_recipients.json    sms_recipients.json
     │
     ▼
send_notification/recipients.py      (read on every send)
```

```
[
  {"username": "...", "email": "a@iliauni.edu.ge", "is_staff": true,  "is_active": true},
  {"username": "...", "email": "b@example.com",    "is_staff": false, "is_active": true}
]
```

```
[
  {"username": "...", "phone_number": "+995555000001", "is_staff": true, "is_active": true}
]
```

- An entry is active only if **both the recipient and the address or number** are active;
- the `staff` group gets only `is_staff: true`; `all` gets every active entry;
- the file is read on every send, so an update takes effect immediately;
- the file changes only after a successful response, atomically (first `.tmp`, then rename);
- if the API is unreachable, **both old files stay**. If one list has no active entry, only that file stays old (to write an empty list — `--allow-empty`);
- log: `logs/update_recipients.log` — `UPDATED`, `UNCHANGED`, `FAILED`.

```
./recipients/update_recipients.py           # update
./recipients/update_recipients.py --dry-run  # show only, files are not changed
```

12. Tests

```
./venv/bin/python -m pytest -q
```

The tests need no database, SMTP or network: every external service is replaced by a fake.

File	What it checks
`test_scalert_event.py`	arguments, channel independence, `DONE`, log format, API data
`test_send_eq2api.py`	create, update, rollback, publish / unpublish, validation, magnitude codes
`test_send_notification.py`, `test_sms.py`	email and SMS text, the provider chain
`test_seiscomp_db.py`	reading from the SeisComP database and retrying
`test_update_recipients.py`	updating the recipient files
`test_config.py`	reading `.env`

13. File layout

```
scalert_event/
├── scalert_event.py          entry point (called by scalert)
├── scalert_event_ka.md       setup guide
├── config.py                 reads .env → Settings
├── seiscomp_db.py            load_event(event_id) -> EventData
├── .env                      passwords and keys (not in git); template — .env.example
├── requirements.txt          pymysql, requests, certifi, pytest
├── automate_events/          decision
│   ├── automate_event.py     process_event, EventAssessment
│   ├── rules.py              who is notified, send_api, publish_iliauni
│   ├── location.py           location text (Georgian and English)
│   ├── acceleration.py       acceleration formula
│   ├── borders_db.py         ies_borders database
│   └── geo.py                distance, direction, polygon
├── send_notification/        email and SMS
│   ├── message.py            subject, email and SMS text
│   ├── recipients.py         recipients
│   ├── mailer.py             SMTP
│   ├── ies_mail_sender.py    send_event_mail
│   └── ies_sms_sender.py     send_event_sms, SmsSender (magti → smsoffice → gsm)
├── send_eq2api/              iesdata site and WordPress
│   ├── event_sender.py       send_event_api (SeisComP → site format)
│   └── send_eq2api.py        EventApiClient
├── recipients/
│   ├── update_recipients.py  updates the recipient files from the API
│   ├── email_recipients.json (not in git); template — email_recipients.example.json
│   └── sms_recipients.json   (not in git); template — sms_recipients.example.json
├── docs/                     this documentation: .md, .html, .pdf, build_docs.py
├── logs/                     event_scalert.log, update_recipients.log
├── tests/
└── venv/                     Python environment (not in git)
```